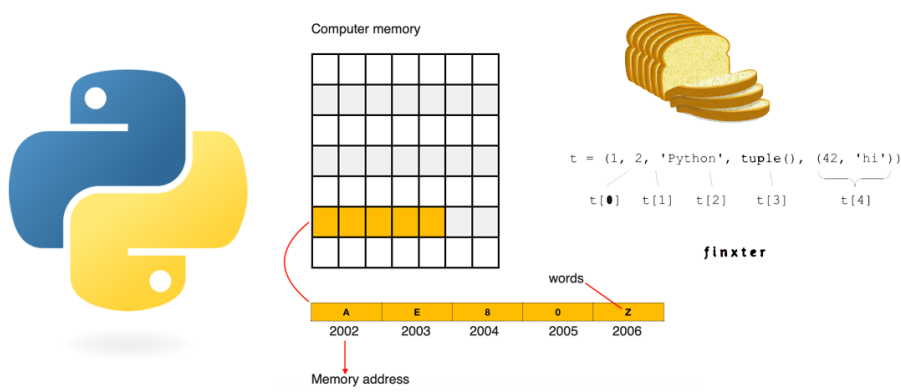


Memory Management in Lists and Tuples

Python has more than one data structure type to save items in an ordered way. This article looks at lists and tuples to create an understanding of their commonalities and the need for two different data structure types. It also looks at how the memory is managed for both of these types. This article is written with reference to CPython implementation.



The commonalities between lists and tuples are:

- Both are sequence data types.
- Each item stored in a list can be of any data type.
- Elements can be accessed by indexing and slicing.
- Both allow duplicates.

Let's see how they are different with examples.

Lists

Lists are mutable in nature, and are sortable. A list can be used to save any kind of object. An example is:

```
# example of list
list1 = [1, 2, "three", 4, 5]
print(type(list1))
print(list1)
```

The output is:

```
<class 'list'>
[1, 2, 'three', 4, 5]
```

Slicing

You can access the contents of a list in the following ways:

```
# access elements using index ; first 2 indexes
print(list1[0:2])
```

```
# access the last 2 indexes
print(list1[-2:])
```

The output is:

```
[1, 2]
[4, 5]
```

Mutable

We can edit the values in the list as follows:

```
# list are mutable, which means we can change the value of
the list
list1[0] = 100
print(type(list1))
print(list1)
```

The output is:

```
<class 'list'>
[100, 2, 'three', 4, 5]
```

Memory allocation

We have now come to the crux of this article — how memory is managed while storing the items in the list. We will first see how much memory is currently allocated, and later see